PROJECT LIFELINE

An off grid resilient communication system for emergency coordination when conventional infrastructure is unavailable.

EN-3 Final Project Report

Prepared by:

Ariyasingha NRTS – 250040V

Bandara DMPI – 250066F

Arnigan Y – 250041B

Arnihan S – 250042E

Department of Electronic & Telecommunication Engineering

University of Moratuwa

10/22/2026

Abstract

Natural disasters can disrupt cellular networks, internet connectivity, and electrical power at the very time when communication is essential resulting in delayed responses and rescue teams not being able to find the individuals that need to be rescued. LifeLine seeks to solve this problem by offering a robust low power emergency communication channel through the use of LoRa (Long Range) technology. LifeLine uses LoRa technology, GPS, and Raspberry Pi to transmit emergency messages as well as location information in cases where communication systems are down. The messages received are then parsed and shown on a map interface to enable rescue teams to find and track the individuals in need. Our ultimate goal is to help rescue teams quickly understand the situation, coordinate their efforts, and respond more effectively to disasters.

Contents

1. Introduction
    a. Problem statement
    b. Our solution
    c. Objectives
    d. Constraints and Assumptions
2. Technical Background
    a. Limitations of currently used communication methods
    b. Why LoRa?
    c. High level system architecture
    d. How does mesh prevent loops?
    e. Hardware
    f. Schematics
    g. Acknowledgement (ACK) and retry protocol
3. User guide for Nodes
4. Conclusion and Future work
5. Deployment readiness and statement
6. Appendix

1. Introduction

Problem statement

Natural disasters can corrupt the regular communication grid. Sometimes disaster relief stations cannot identify who and where are the survivors, is there any other secondary disaster happened in those disaster isolated areas.

As an example when Sri Lankan people faced **Ditwah,** Out of roughly 16,000 mobile transmission towers nationwide, **over 4,000 main towers went completely inactive**. Millions of users across districts like Nuwara Eliya, Badulla, Kandy, Puttalam, and Vavuniya were left completely isolated without cellular or internet connectivity, severely hampering emergency coordination. Even some of our closest friends also unabled to get informed about their family.

What is our solution?

Our solution is distribute a device called **Node** around a country and keep them online. When a part of the district got isolated people can connect with the nearest Node with their phone and send SOS message to nearby disaster relief center. Node uses radio mesh network to reach further distances. So that network cannot be isolated easily.

Objectives

- Develop a node architecture using a suitable microcontroller and a radio transceiver.
- Implement message addressing, message identifiers, location sensor data extractions.
- Creating a user friendly mobile application and displaying delivery status.
- Developing a mesh network that do not repeat a successfully sent message.
- Repeatable field test procedure

Constraints and assumptions

- Performance depends on the environment whether it is a plain or terrain, urban or rural and the number of active Nodes.
- Nodes are assumed to be operated by trained users and they do not supply wrong information.
- It is assumed that users will not misuse these legally un-logged and uninterrupted communication systems.

2. Technical Background

Limitations of currently used communication methods

Communication systems range from public cellular networks and satellite systems to professional land mobile radio (Walkie talkie), armature radio (ham radio), pagers etc.

Cellular networks are vulnerable for majority of disasters. They could be easily go down. Satellite systems require compatible equipment, clear sky view for many services, subscription or airtime arrangements, and additional cost.

Professional radio systems are robust but requires licensed operators, repeaters and expensive equipment.

A small radio mesh that controlled with a user friendly application in their phone can be a good solution in this case.

Why LoRa?

LoRa (Long Range) is a technology developed for send small amount of data over long distances using very little power.

Here are some rough ranging data of LoRa according to environment

- **Rural / Open Countryside: 10 km to 20+ km**
- **Suburban Areas: 5 km to 10+ km**
- **Dense Urban / Smart City: 1 km to 5 km**
- **Mountainous / Forested Terrain: 5 km to 10 km**
- **Indoor / Multi-wall Buildings: 100 m to 1 km**

High-Level System Architecture

When a user send a sos message through their mobile phone, the application formats the message and combines users current location with the GPS sensor in their phone. And send it via BLE to the microcontroller.

Microcontroller forwards the exact BLE message with the LoRa module to the air.

Case 1

If the disaster relief station is in the range of the Node operator, then the SOS directly hit the LoRa transceiver in the Command Center device in disaster relief station.

Case 2

If the disaster relief center is not in the range and if there is another Node in the range then the data packet hits that LoRa module, check where it come from and add the data packet where it landed and again release it into the air. And this process repeats until the data packet hit the command center.

Finaly after the data packet received to the command center computer, it processes the data packet and extract where it comes from, what is the emergency type, and updates it in the map window. Then it sends a message that the sos has been received. That message also hops between other nodes and hit the target node.

How does mesh prevent loops?

When a sos received to a node, then it checks the node history in the message (during hopping the current node number is combined to the message). If the current node in the node history it means the same message previously transmitted by the node. So it ignores the message.

If the same message (detects by checking the time in the message) came from two separate paths, then that message also been transmitted by the node. So that also ignored.

When the node received a message and if the distance from the disaster relief center and the coordinates in the message (the origin) is lower than the distance between the current node and the disaster relief center then the message get ignored. Because the message is transmitting away from the destination.

Hardware

Field Node

• ESP32 microcontroller • Ra-02 SX1278 433MHz LoRa module	For handle Bluetooth connectivity and control LoRa module For long range radio communication and reduce the cost

Command center (Central Node)

• Raspberry Pi • Ra-02 SX1278 433MHz LoRa module	For handle massive number of data coming from hundreds of field nodes. And make the command center device portable.

Pin Configurations

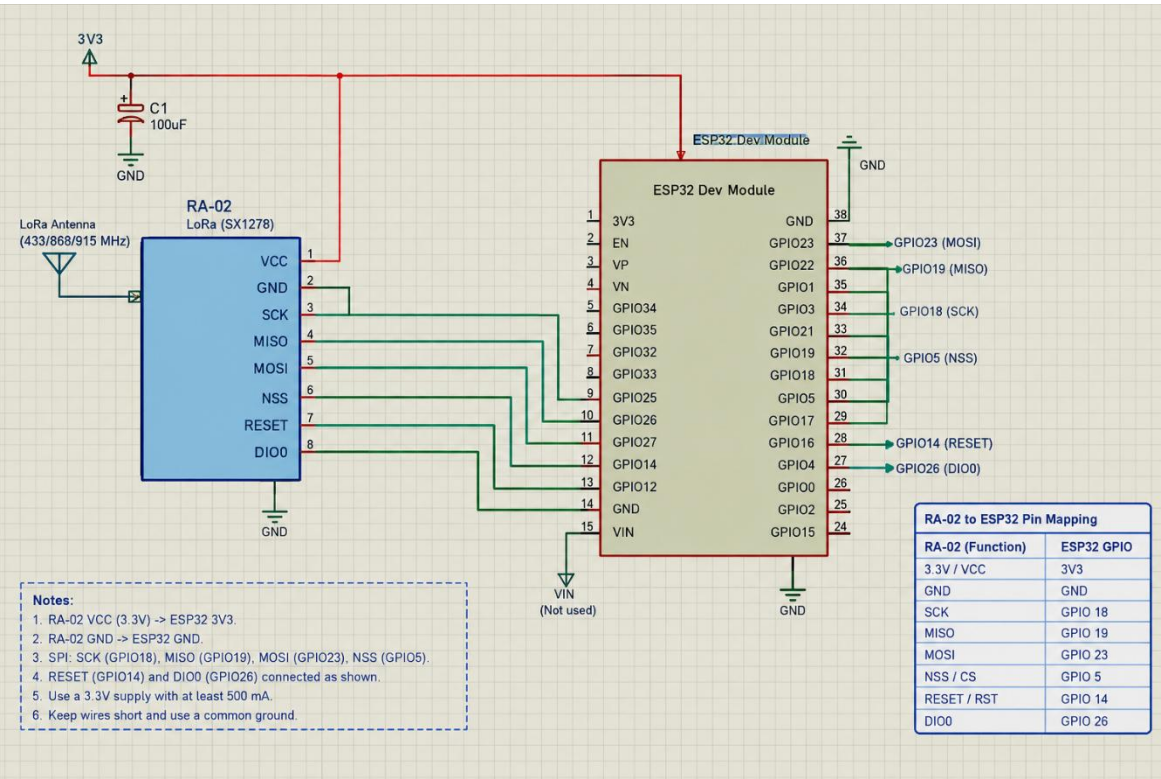
RA-02 LoRa to ESP32 (Survivor / Field Nodes)

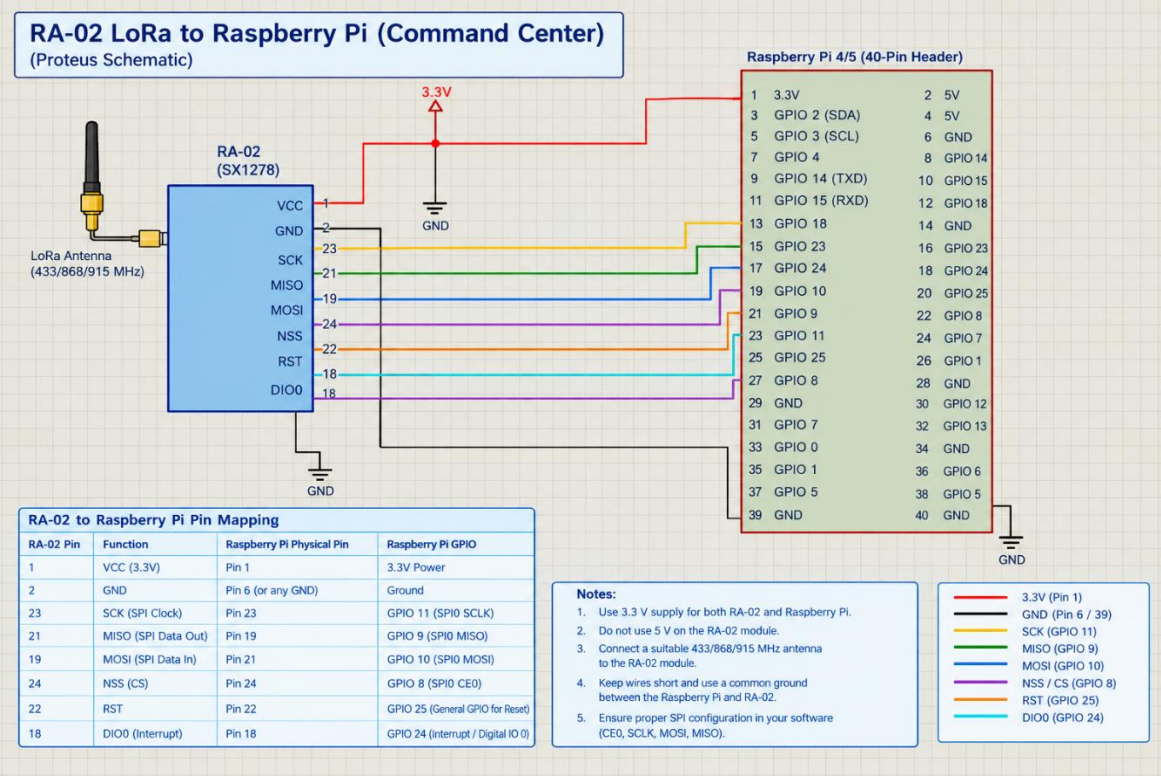
RA-02	Function	ESP32
3.3V / VCC	Power	3V3
GND	Ground	GND
SCK	SPI clock	GPIO 18
MISO	SPI data out	GPIO 19
MOSI	SPI data in	GPIO 23
NSS / CS	SPI chip select	GPIO 5
RESET / RST	Radio reset	GPIO 14
DIO0	Interrupt	GPIO 26

RA-02 LoRa to Raspberry Pi (Command Center)

RA-02 Pin	Function	Raspberry Pi Physical Pin	Raspberry Pi GPIO / Function
VCC	3.3V Power	Pin 1	3.3V Power
GND	Ground	Pin 6 (or 9, 14, 20, 25, 30, 34, 39)	Ground
SCK	SPI Clock	Pin 23	GPIO 11 (SPI0 SCLK)
MISO	SPI Data Out	Pin 21	GPIO 9 (SPI0 MISO)
MOSI	SPI Data In	Pin 19	GPIO 10 (SPI0 MOSI)
NSS (CS)	SPI Chip Select	Pin 24	GPIO 8 (SPI0 CE0)
RST	Reset	Pin 22	GPIO 25 (General GPIO for Reset)
DIO0	Interrupt	Pin 18	GPIO 24 (Interrupt / Digital IO 0)

Schematics


3V3
C1
100uF
GND
RA-02
LoRa (SX1278)
LoRa Antenna
(433/868/915 MHz)
VCC
GND
SCK
MISO
MOSI
NSS
RESET
DIO0
ESP32 Dev Module
3V3
EN
VP
VN
GPIO34
GPIO35
GPIO32
GPIO33
GPIO25
GPIO26
GPIO27
GPIO14
GPIO12
GND
VIN
GPIO23
GPIO22
GPIO1
GPIO3
GPIO21
GPIO19
GPIO18
GPIO5
GPIO17
GPIO16
GPIO4
GPIO0
GPIO2
GPIO15
GPIO23 (MOSI)
GPIO19 (MISO)
GPIO18 (SCK)
GPIO5 (NSS)
GPIO14 (RESET)
GPIO26 (DIO0)
VIN
(Not used)
Notes:
1. RA-02 VCC (3.3V) -> ESP32 3V3.
2. RA-02 GND -> ESP32 GND.
3. SPI: SCK (GPIO18), MISO (GPIO19), MOSI (GPIO23), NSS (GPIO5).
4. RESET (GPIO14) and DIO0 (GPIO26) connected as shown.
5. Use a 3.3V supply with at least 500 mA.
6. Keep wires short and use a common ground.
RA-02 to ESP32 Pin Mapping
RA-02 (Function) | ESP32 GPIO
3.3V / VCC | 3V3
GND | GND
SCK | GPIO 18
MISO | GPIO 19
MOSI | GPIO 23
NSS / CS | GPIO 5
RESET / RST | GPIO 14
DIO0 | GPIO 26


RA-02 LoRa to Raspberry Pi (Command Center)
(Proteus Schematic)
3.3V
GND
RA-02
(SX1278)
LoRa Antenna
(433/868/915 MHz)
VCC
GND
SCK
MISO
MOSI
NSS
RST
DIO0
Raspberry Pi 4/5 (40-Pin Header)
1 3.3V
3 GPIO 2 (SDA)
5 GPIO 3 (SCL)
7 GPIO 4
9 GPIO 14 (TXD)
11 GPIO 15 (RXD)
13 GPIO 18
15 GPIO 23
17 GPIO 24
19 GPIO 10
21 GPIO 9
23 GPIO 11
25 GPIO 25
27 GPIO 8
29 GND
31 GPIO 7
33 GPIO 0
35 GPIO 1
37 GPIO 5
39 GND
2 5V
4 5V
6 GND
8 GPIO 14
10 GPIO 15
12 GPIO 18
14 GND
16 GPIO 23
18 GPIO 24
20 GPIO 25
22 GPIO 8
24 GPIO 7
26 GPIO 1
28 GND
30 GPIO 12
32 GPIO 13
34 GND
36 GPIO 6
38 GPIO 5
40 GND
RA-02 to Raspberry Pi Pin Mapping
RA-02 Pin | Function | Raspberry Pi Physical Pin | Raspberry Pi GPIO
1 | VCC (3.3V) | Pin 1 | 3.3V Power
2 | GND | Pin 6 (or any GND) | Ground
23 | SCK (SPI Clock) | Pin 23 | GPIO 11 (SPI0 SCLK)
21 | MISO (SPI Data Out) | Pin 19 | GPIO 9 (SPI0 MISO)
19 | MOSI (SPI Data In) | Pin 21 | GPIO 10 (SPI0 MOSI)
24 | NSS (CS) | Pin 24 | GPIO 8 (SPI0 CE0)
22 | RST | Pin 22 | GPIO 25 (General GPIO for Reset)
18 | DIO0 (Interrupt) | Pin 18 | GPIO 24 (Interrupt / Digital IO 0)
Notes:
1. Use 3.3 V supply for both RA-02 and Raspberry Pi.
2. Do not use 5 V on the RA-02 module.
3. Connect a suitable 433/868/915 MHz antenna to the RA-02 module.
4. Keep wires short and use a common ground between the Raspberry Pi and RA-02.
5. Ensure proper SPI configuration in your software (CE0, SCLK, MOSI, MISO).
3.3V (Pin 1)
GND (Pin 6 / 39)
SCK (GPIO 11)
MISO (GPIO 9)
MOSI (GPIO 10)
NSS / CS (GPIO 8)
RST (GPIO 25)
DIO0 (GPIO 24)

The Acknowledgement (ACK) and Retry Protocol

After a message sent to the command center server, then the server broadcasts the NIC number + message number (hashed and encrypted) back. If the origin node of the message received that it checks the hashed and encrypted NIC number + message number, then the node knows the previous message has been sent successfully.

Example

SOS Message format:

<Node chain>: [SOS] : <message id> : <timestamp> | <alert> | GPS: <lat, lon>

ACK Message format:

<Node chain>: [ACK] : <a base85 string of the sha256 hash of the current users NIC number with the message number>

We use the SHA256 hashing to secure the NIC and base85 encryption to shorten the hash.

Until the ACK message receive the Node continuously send the same message with 5-minute intervals. After the acknowledgement that retrying stops.

This protocol is implemented to ensure the data reached destination safely.

3. User guide for Nodes

- User should have the LifeLine application in their mobile phone
- First Log in to your application by adding your NIC number.
- Connect to the Node with the application. The node is shown as LifeLine_Node_XXX
- After a successful connection the user can select the emergency type and click send button.
- And wait for the message sent notification. **Do not** try to click more and more times if the notification did not came. The device will repeat your message until a valid response.

4. Conclusion and Future work

Conclusion

Through this project we defined a low-cost, off-grid disaster communication mesh concept based on locally powered radio nodes, short text messages, and store-and-forward relaying. It establishes a requirements-driven architecture, outlines hardware and firmware responsibilities, and proposes a verification plan covering functional behavior, multi-hop delivery, latency, range, power consumption, and recovery after link failure.

Future work

- Build and test at least 5 nodes to demonstrate a genuine multi-hop delivery.
- Add a rechargeable LiPo battery and design a cover for field nodes for improve durability and portability.
- Implement a user authentication system and payload encryption with a documented key-management process.

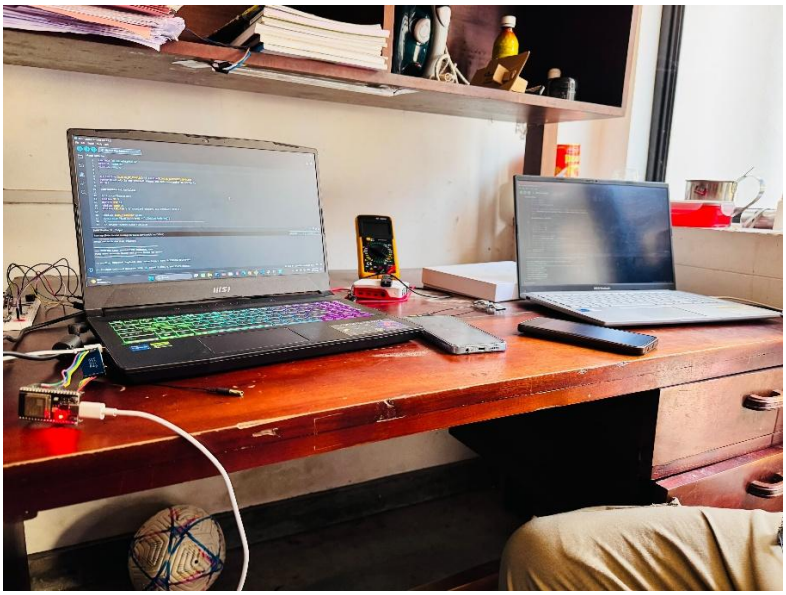
- Evaluate controlled flooding against a routing approach under increasing traffic load.
- Add accessible UI features such as high-contrast status, clear acknowledgement states, vibration alerts, and local-language labels.
- Conduct user exercises with emergency-response stakeholders and document the limits of the prototype.

5. Deployment readiness and statement

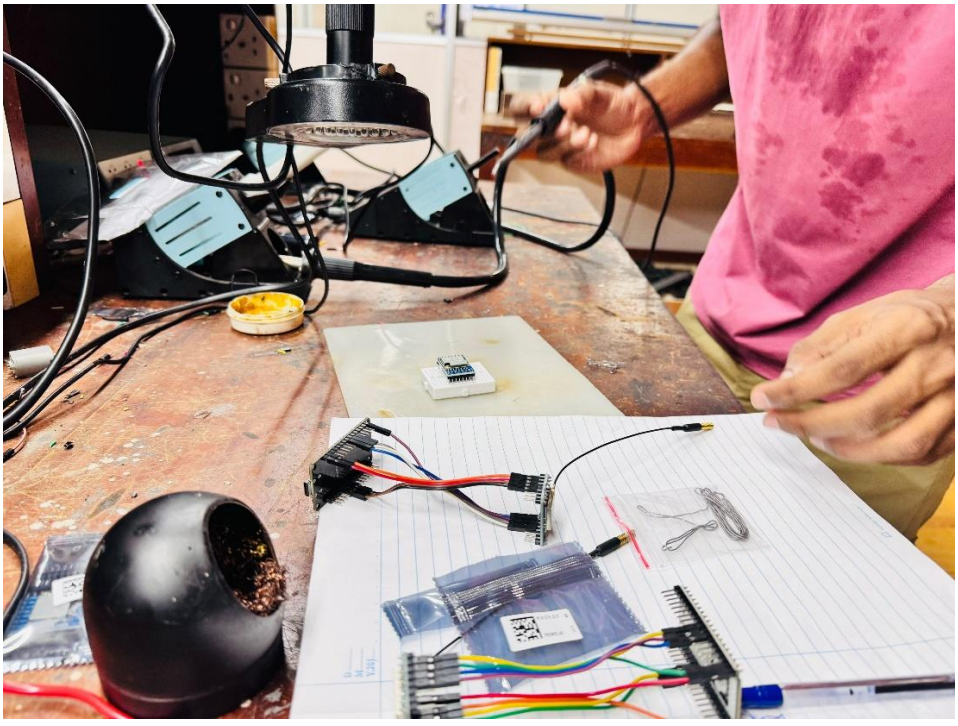
A student prototype should not be described as disaster-ready solely because messages pass between two boards. Before any real emergency deployment, it would require radio-regulatory review, security review, robust enclosure and power testing, documented operating procedures, representative field exercises, and independent validation. It should remain supplementary to established emergency communication systems.

6. Appendix

Early stages of our project







How it finished



View our project on GitHub:

https://github.com/ThenulSahansa/LifeLine/